

PHASE 1, MODULE 11: OBJECT-ORIENTED PROGRAMMING (OOP)

Building Industrial-Grade Blueprints

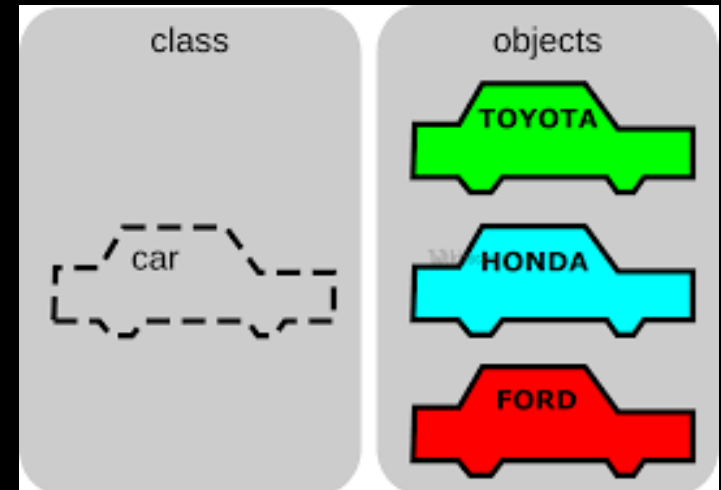
Until now, we have been writing code as a series of steps. Today, we shift our mindset. We will learn how to group data and behavior together into 'Objects.' This is how large-scale AI frameworks like PyTorch and TensorFlow are built.

Presented By: Zaryab Ahmad

What is a Class? (The Blueprint)

Blueprints vs. Objects

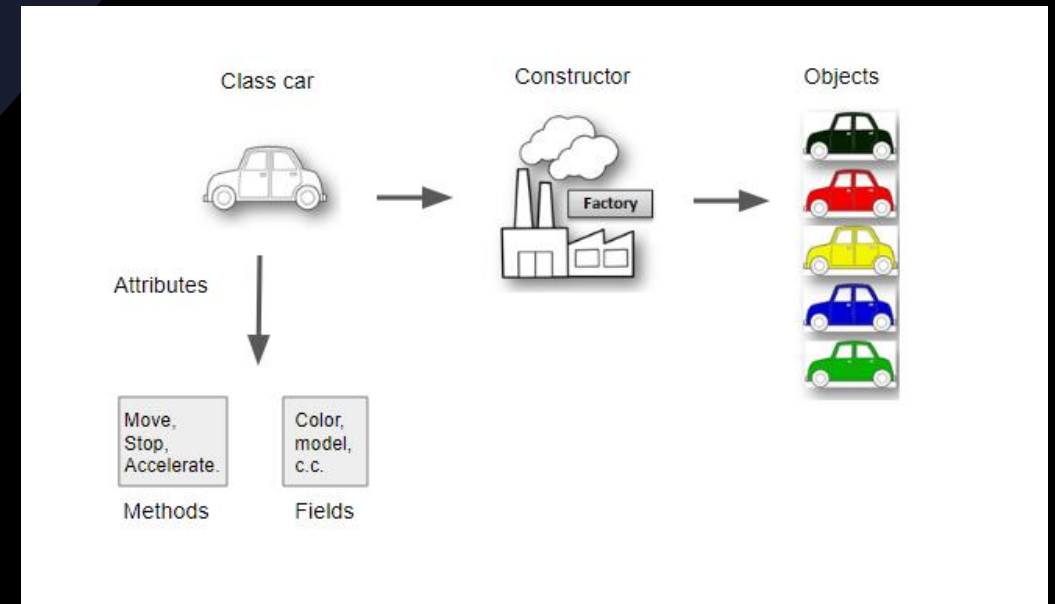
- **Class:** The architectural drawing (e.g., a "Car" design).
- **Object (Instance):** The physical item built from that drawing (e.g., your specific Car).
- **The Goal:** Write the logic once in a Class, then create 1,000 unique Objects from it.



Attributes and Methods

State and Behavior

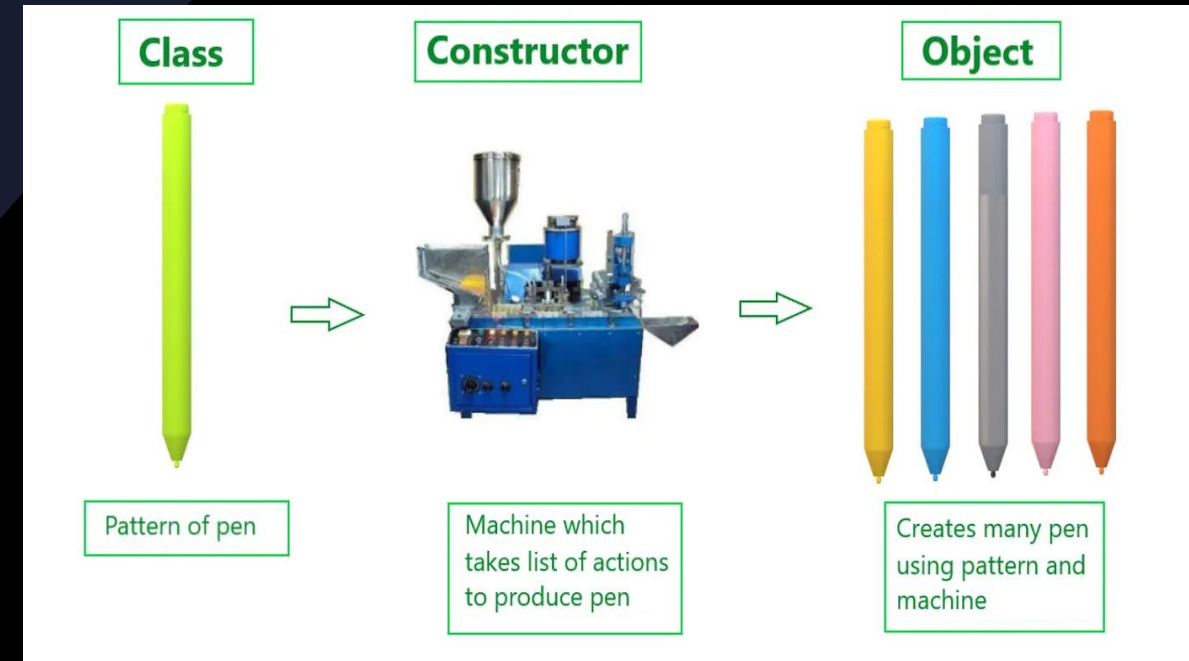
- **Attributes (Data):** What the object is (e.g., color, model, speed).
- **Methods (Functions):** What the object does (e.g., start_engine, accelerate).
- *self*: The keyword that allows an object to "talk" to its own data.



The Constructor (`__init__`)

The Birth of an Object

- The `__init__` method runs automatically the microsecond an object is created.
- It sets the initial state of the object (e.g., setting the patient ID or loading the AI model weights).



Inheritance (Professional Hierarchy)

Don't Rebuild, Inherit

- A "Child" class can take all the features of a "Parent" class.
- **Use Case:** A generic Sensor [?]
class (Parent) A TemperatureSensor class (Child).

